Energy-Based CNN Pruning for Remote Sensing Scene Classification

Yiheng Lu, Maoguo Gong, *Senior Member, IEEE*, Zhuping Hu, Wei Zhao, *Senior Member, IEEE*, Ziyu Guan, *Senior Member, IEEE*, and Mingyang Zhang, *Member, IEEE*

***Abstract*— Convolutional neural networks (CNNs) have been adopted to classify the remote sensing scene image. However, the application of these complicated networks on the satellite platform is difficult because of the limited computation resources. Therefore, we propose an energy-based filter pruning framework (EFPF) to reduce the size of the original model. The energy can be obtained through the eigenvalues of each weight tensor by singular value decomposition (SVD). Specifically, we calculate the energy of each layer by the ratio of eigenvalues that are lower than a specified truncation parameter and then remove filters from the original layer in light of the degree of energy. The EFPF is reliable, because SVD techniques can capture the covariance among all filters from the original weight tensor, and therefore, the energy from the eigenvalues can reflect the redundancy of the filters (i.e., if the distribution of eigenvalues is sharp, then the energy among filters will be lower, and the redundancy will be higher). Surprisingly, the EFPF can reduce the floating point operations (FLOPs) and parameters, as well as improve the top1 accuracy obviously when the VGG-16 and ResNet-50 are adopted to classify the AID, NWPU45, PatternNet, and WHU19 datasets. Additionally, the EFPF can achieve similar pruning results when the original model is fully trained (converge) and undertrained (In-converge), which means that we can save the training computation resources for the original model.**

***Index Terms*— Convolutional neural network (CNN), energy-based, filter pruning, remote sensing scene image classification.**

I. Introduction

DUE to the thriving of satellite sensor technology, the remote sensing scene classification has been used broadly in military deployment [1], urban planning [2], traffic flow prediction [3], and other fields. Recently, the remote sensing scene image classification task resorts the convolutional neural networks (CNNs) to improve the classification ability [4], [5], [6], [7].

However, these hand-designed networks are over-parameterized with scale-up size, and it may lead to over-fitting problems, as well as burdens the computation hardware due to the heavy footprint on parameters and floating point operations (FLOPs), especially for the remote sensing images, because the resource of the satellite platform is explicitly limited.

Therefore, pruning and neural architecture searching (NAS) methods are used to compress the models. Specifically, pruning techniques can help customize a compact subnetwork by removing trivial parameters or filters from the original model, and NAS can also yield a compact model with high representation power by combining filters automatically from bottom to top. Real et al. [8] and Zoph and Le [9] use the evolutionary algorithm and reinforce learning to search for better structure for a specified dataset, respectively. Shahriari et al. [10] propose the "Bayesian optimization" method to predict a reasonable architecture. Differently, Liu et al. [11] incorporate the discrete searching space of networks into continuous space by the softmax, which can speed up the searching process. Even NAS-based methods can achieve powerful representation ability with the lower complexity of the models; the search costs of the NAS are pretty higher, which means that we need to prepare massive hardware to search for an appropriate model. Therefore, we aim to optimize the structure of the models by pruning techniques.

There are two prevalent pruning methods that are resorted to saving computation resources on the satellite platform, that is unstructured and structured pruning. In particular, unstructured pruning methods try to remove partial parameters [12], [13], [14], which can reduce the consumption of parameters but cannot eliminate the FLOPs' occupation. It is because according to the existing hardware situation, unstructured pruning is more inclined to zero parameters of convolutional kernel than to be deleted. The zero setting parameters still exist in the network structure, so the amount of FLOPs will not decrease. Structured pruning can reduce the parameters and FLOPs, because the intact filters will be removed, as well as maintain the representation ability of the slimmed models [15], [16], [17].

Nevertheless, previous structured pruning methods are weight sensitive, which implies that we cannot obtain reliable pruning results when the original model is not trained well. Considering the constraints of satellite resources, we propose the energy-based filter pruning framework (EFPF) to customize submodels by removing filters from original models. It can outperform classical pruning methods, as well as achieve comparable results as the NAS-based methods on remote sensing images. Moreover, the EFPF can yield impressive results even when the original model is not trained fully and then save the training time of the original model on the satellite platform.

Manuscript received 2 November 2022; revised 20 February 2023, 7 April 2023, and 18 April 2023; accepted 8 May 2023. Date of publication 17 May 2023; date of current version 1 June 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62036006, Grant 62133012, and Grant 61936006; and in part by the Innovation Capability Support Program of Shaanxi under Grant 2021TD-05. *(Corresponding author: Maoguo Gong.)*

The authors are with the State Key Laboratory of Collaborative Intelligence Systems, Ministry of Education, Xidian University, Xi'an 710071, China (e-mail: lyhxdu@gmail.com; gong@ieee.org; huzhuping@stu.xidian.edu.cn; ywzhao@mail.xidian.edu.cn; ziyuguan@nwu.edu.cn; omegazhangmy@gmail.com).

Digital Object Identifier 10.1109/TGRS.2023.3277077

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

For the remote sensing scene datasets, the core of the EFPF lies in the redundancy measurement of filters by the energy that is obtained from the eigenvalues for each convolutional layer. Usually, the weight tensor from the original convolutional layer will be decomposed by singular value decomposition (SVD) to generate an eigenvalue matrix, and the original works for the SVD aim to reduce the complexity of the original weight tensor or feature maps by the matrix approximation, which may damage the ability of the model obviously [18], [19], [20]. Instead, we use the distribution of all eigenvalues to reflect the energy for measuring redundancy among filters. Moreover, we will set a truncation parameter α to calculate the energy of each layer by the ratio of eigenvalues that are lower than α. Consequently, we will remove several filters from the original convolutional layer by the energy.

It is trustworthy that the decomposition of the original weight tensor by SVD can capture the covariance among all filters, and then the eigenvalues in the low-rank space can be used to measure the redundancy within the filters, i.e., if the shape of the distribution for eigenvalues is sharp, only a few eigenvalues will account for the majority of information, and then higher redundancy of filters can be observed from the original convolutional layer. Namely, the energy of the original layer is lower when the shape of eigenvalues is sharp. Therefore, EFPF is reliable to slim the original network.

Usually, the weight tensor of each convolutional layer is packed as a 4-D tensor and the straight decomposition of it is difficult. As a result, we will fold the 4-D tensor into a 2-D matrix with two methods, i.e., the average parameter distribution (a-PD) and maximum parameter distribution (m-PD). Here, the a-PD strategy means that we will fold the original weight tensor with the average value along the dimension of filter width and height, and m-PD implies that we will fold the original matrix with the maximum value along the width and height in light of filter dimension. After the folding of each weight tensor, we can specify an appropriate truncation parameter to remove filters in proportion.

In order to make each convolutional layer comparable at the same magnitude, we utilize the "0–1" normalization to constrain each eigenvalue between 0 and 1. Then, the distribution of normalized eigenvalues for each convolutional layer can contribute to measuring the redundancy among filters. It is obvious that the distribution of normalized eigenvalues maintains the same properties as the original eigenvalues (i.e., the sharper distribution, the higher redundancy). Therefore, we can seduce that all the eigenvalues are distributed between 0 and 1 and then can achieve global pruning when we set the same truncation parameter for all layers to calculate the corresponding energy. Incredibly, we can reduce the parameters and FLOPs by 73.15% and 32.15% for VGG-16 on NWPU45 [21], as well as improve baseline accuracy by 1% when the original VGG-16 is fully trained.

However, we also illustrate that EFPF is robust to the update of the parameters, and it means that even when the original model is only trained with a few epochs, we can still obtain similar pruning results as the model that has been fully trained. The consistency of the EFPF is verified by mathematical proofs and extensive experiments.

Three contributions are summarized in this article as follows.

1) We propose the EFPF with a-PD and m-PD to measure the redundancy of filters for each convolutional layer, and a truncation parameter is regulated to achieve global pruning when the original layers' weight tensor is projected into low-rank space. EFPF outperforms the classical global filter pruning methods, as well as achieves comparable results as the NAS-based method with lower searching costs.
2) Our method can achieve similar pruning results when the original model is trained with a few epochs; then, we can save the training computation resources for the original model when the EFPF is used to compress the CNN on the satellite platform.
3) Impressive pruning results are accomplished on VGG-16 and ResNet-50 with AID [22], NWPU45 [21], PatterNet [23], and WHU19 [24], including the reductions of FLOPs and parameters, as well as including the improvement of top1 accuracy.

This article is written as follows. In Section II, we explore the related works about the classification tasks for remote sensing images, as well as introduce the pruning-based and NAS-based methods. In Section III, we illustrate the superiority of our pruning framework. In Section IV, we verify our methods on different models and datasets. In the end, we summarize the work as a whole.

II. Related Work

In this part, we will introduce the classification work about the remote sensing scene image using the neural networks, as well as illustrate the compression techniques for reducing the computation complexity for the above-mentioned networks. Specifically, the pruning techniques and NAS techniques will be exploited explicitly in the following parts.

A. Classification Task for the Remote Sensing Scene

Originally, there are several surveys for remote sensing image processing [25], [26], [27]. With the development of deep techniques, more works have been mentioned in the later surveys [28], [29], [30]. Intuitively, classical architectures such as AlexNet [31], VGGNets [32] and ResNet [33] are used to tackle the scene classification tasks.

Recently, Liu et al. [34] propose a Siamese model that combines two identical traditional models to share the pre-trained parameters, and then improves the performance of the models on remote sensing images. Zhu et al. [35] propose an attention-based CNN model to focus on the gradient-weighted class activation mapping (Grad-CAM), as well as introduce a multiplicative fusion of deep features and the center-based cross-entropy loss function to fix the problem of intra-class diversity and repeated texture within remote sensing images. Gong et al. [36] classify scenes by using attention mechanism. Jiang et al. [37] used the method of self-supervised learning to classify scenes. Zhang et al. [38] put forward a "CNN-CapsNet" that combines a pre-trained CNN and a CapsNet. By replacing the fully connected layer

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

of CNNs with CapsNet, the "CNN-CpasNet" can achieve better classification performance. Minetto et al. [39] present a "Hydra" method to create a coarsely optimized CNN by initializing a good starting pointing for further optimization, and then, the weights are updated several times to construct an ensemble of CNNs for describing the Hydra's heads. Simultaneously, Chaib et al. [40] improve the performance of remote sensing scene classification by deep feature fusion. He et al. [41] and Lu et al. [42] design different mechanisms to improve the representation ability of CNNs by using multilayer stacked covariance pooling and feature aggregation, respectively.

However, other useful CNN-based models are applied to improve the classification accuracy of the remote sensing images [43], [44], [45], [46], [47]. Notably, these models are designed by prior expert knowledge, which means that their structure may not adapt well for the specified datasets, as well as burden for the hardware. Consequently, the pruning-based and NAS-based methods are explored to customize a suitable architecture for the dataset.

B. Pruning-Based Techniques

Pruning techniques can reduce the size of hand-designed models by removing weight parameters or intact filters, and they are named unstructured pruning and structured pruning. The unstructured pruning methods dominate the pruning task before the prevalence of CNNs [12], [13], [48]. These methods can perform well on neural networks, because the previous networks are designed with simple architectures.

Nevertheless, unstructured methods cannot reduce the FLOPs for the CNNs, because the number of feature maps cannot be eliminated. Instead, structured pruning methods are proposed to both reduce the FLOPs and parameters on CNNs by removing filters. Li et al. [49] use the L_1-norm to evaluate the importance of filters. Liu et al. [16] remove filters by adding the constraints on the BatchNorm layers, and the filters will be pruned if the corresponding batch neurons are relatively small. Molchanov et al. [15] propose the "Taylor Pruning" by decomposing the complicated pruning process as a polynomial at first-order, which can simplify the evaluation of each filter as the product of weight value and gradient for the features.

Additionally, there are several structured pruning methods proposed to compress the models by removing filters [17], [50], [51]. However, these methods are sensitive to the update of weight, and it implies that they cannot perform well if the original model is trained well. Therefore, we propose the EFPF to reduce the size of the original models, as well as avoid interference with the update of the individual parameter from the original model via SVD. Originally, the SVD is utilized to reduce the computation complexity of neural networks by low-rank approximation. Tai et al. [18] and Zhang et al. [19] use the SVD and generalized singular value decomposition (GSVD) to achieve the low-rank approximation of feature maps for reducing the computation consumption, respectively. Huang et al. propose the low-complexity probabilistic autoencoder neural network to initialize the features of recommendation systems. However, these methods focus on the decomposition of the original weight tensor or features while ignoring the effect of filters, and the approximation of matrices usually degenerates the performance of the model. Instead, EFPF adopts the SVD to reflect the contributions of filters and then to remove filters for reducing the size of the original model, as well as to maintain a higher representation ability.

C. NAS-Based Techniques

NAS aims to search for a good structure from bottom to top. Zheng et al. [52] propose an AutoCL method to search for a better structure by designing a hierarchical search space. Jing et al. [53] propose the AutoRSISC algorithm to reduce the redundancy in the search space by sampling the neural architecture in a certain proportion with continuous relaxation. Peng et al. [54] design a continuous searching space by a greedy and progressive strategy, as well as introduce bilevel optimization. Additionally, [56] and [57] also propose different NAS-based methods to improve the classification accuracy of the customized models, as well as reduce the size of the models. However, the search process is cumbersome when the dataset is large, which requires a lot of computation resources to run the algorithm. Therefore, we utilize pruning techniques to remove filters from the human-designed models. Then, we can also compress the model from the perspective of FLOPs and parameters without the heavy searching costs.

III. Methodology

In this article, we propose the EFPF to reduce the size of CNNs by removing trivial filters from the original convolutional layer. The flow of our pruning framework can be depicted in Fig. 1. The core of this method lies in the redundancy measure of eigenvalues in the low-rank space. For each convolutional layer, the weight tensor will be decomposed by SVD. Then, a truncation parameter α will be applied to calculate how many filters are trivial when their values are smaller than α. Finally, we calculate the energy by leveraging of the ratio of useless eigenvalues within a specified eigenvalue matrix, and then remove filters from the corresponding layers in accordance with the energy. It is proven that the ability of the network is dominated by the structure rather than the weight information [57], and therefore, we will remove filters randomly from each layer.

It is highly believed that if the distribution of the weight parameters is sharper, the energy of the layer is lower, because only a few parameters occupy the majority of useful information. Then, more redundancy will be confirmed, because the representation ability of each convolutional layer has a relation to the distribution of parameters for the filters [58]. Motivated by this, we try to design an energy-based pruning mechanism to compress the model from the perspective of the distribution of parameters within each weight tensor. It should be noticed that the straight analysis of the weight tensor is difficult, because there are too many parameters. Instead, we measure the redundancy of filters by the strength of eigenvalues in low-rank space when the original weight tensor is decomposed through the SVD.

Usually, the SVD can capture the covariance for the original tensor. With respect to the weight tensor of each convolutional

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

layer, the SVD can obtain the covariance among filters and then measure the redundancy by the strength of eigenvalues in low-rank space, i.e., if only a small number of eigenvalues account for the majority of elements for the whole eigenvalues, there may be heavy redundancy of filters in the original convolutional layer. Therefore, EFPF is reliable to prune the original network.

A. Parameter Distribution of Convolutional Layers

For each convolutional layer, the parameters are packed by a 4-D tensor $\mathbf{M} \in \mathbb{R}^{\mathcal{I}\times\mathcal{O}\times\mathcal{H}\times\mathcal{W}}$, where $\mathcal{I}$, $\mathcal{O}$, $\mathcal{H}$, and $\mathcal{W}$ stand for the number of input features, and output features, as well as the height and width for each filter. Let us denote each parameter within $\mathbf{M}$ as w; then, we can define the distribution of $\mathbf{M}$ as

$$\begin{aligned} &\mathbf{M}^{(\mathcal{I}\times\mathcal{O}\times H\times W)} \sim N(n, w) \\ &\text{s.t. } n = \mathcal{I}\times\mathcal{O}\times\mathcal{H}\times\mathcal{W} \end{aligned} \tag{1}$$

where $N(n, w)$ means that the distribution of a weight tensor is parameterized by the n and w, and n stands for the number of parameters within the $\mathbf{M}$. Here, we will authorize the layer with higher energy (importance) if the distribution of parameters is flat. It can be explained that if the distribution of parameters is flat, we will declare that the filters tend to show strong similarity within a layer. Then, it is difficult to identify the redundancy among the filters. In the contrast, trivial filters appear when the distribution of parameters for the weight tensor is sharp, and then we can remove more filters from the corresponding convolutional layer.

The degree of importance d for each convolutional layer can be written as

$$d\left(\mathbf{M}^{\mathcal{I}\times\mathcal{O}\times\mathcal{H}\times\mathcal{W}}\right) = \widetilde{N}(n, w) \tag{2}$$

where the hat of "$\widetilde{N}$" means that the shape of distribution $N(n, w)$ will contribute to evaluating the layer importance. However, the straight analysis of d is difficult, because there may be tremendous parameters within a layer; e.g., there are 1×10^7 parameters in the weight tensor of the last convolutional layer for VGG-16 and, therefore, prevent us analyzing the importance of each layer.

Instead, we utilize the principal component analysis (PCA) to project $\mathbf{M}$ into low-rank space to simplify the analysis of parameter distribution by studying the eigenvalues. Generally, the SVD is an efficient technique to achieve the PCA, as well as capture the covariance among parameters for the original matrix. Therefore, we adopt the SVD to analyze the covariance among filters and then measure the redundancy for the specified layer.

B. Low-Rank Decomposition for Convolutional Layers

In this article, we use the SVD to decompose $\mathbf{M}$ into the low-rank space. Notably, the original $\mathbf{M}$ is packed as a 4-D tensor, and it is hard to decompose a tensor by the SVD straightly. Therefore, we will fold $\mathbf{M}^{\mathcal{I}\times\mathcal{O}\times\mathcal{H}\times\mathcal{W}}$ as a 2-D matrix along the size of the filters. There are two ways to downsize the dimension of $\mathbf{M}$ along the height and width through the average and maximum folding, respectively.

1) Average Folding: The average folding means that the average value of $\mathcal{H}\times\mathcal{W}$ will be used to express the property of the filters. The average folding can be noted as

$$\begin{aligned} \mathcal{H}^{\text{aver}} &= \frac{\sum_h \mathbf{M}[:, :, h, :]}{\mathcal{H}} \\ \mathbf{M}^{\text{aver}} &= \frac{\sum_w \mathcal{H}^{\text{aver}}[:, :, w]}{\mathcal{W}}. \end{aligned} \tag{3}$$

Notably, the dimension of $\mathbf{M}^{\mathcal{I}\times\mathcal{O}\times\mathcal{H}\times\mathcal{W}}$ will be reduced progressively after every folding along one dimension. Consequently, we can get a folded matrix $\mathbf{M}^{\text{aver}}$ that satisfies $\mathbf{M}^{\text{aver}} \in \mathbb{R}^{\mathcal{I}\times\mathcal{O}}$. We name this strategy the a-PD.

2) Maximum Folding: Same as the average folding, we aim to reduce the $\mathbf{M}^{\mathcal{I}\times\mathcal{O}\times\mathcal{H}\times\mathcal{W}}$ to two dimensions with the maximum folding along the height and the width. The maximum folding can be written as

$$\begin{aligned} \mathcal{H}^{\max} &= \text{Max}(\mathbf{M}[:, :, \mathcal{H}, :]) \\ \mathbf{M}^{\max} &= \text{Max}(\mathcal{H}^{\max}[:, :, \mathcal{W}]) \end{aligned} \tag{4}$$

where $\text{Max}(\cdot)$ will calculate the maximum value from a specified tensor or matrix. Then, the original tensor $\mathbf{M}$ can be rewritten as $\mathbf{M}^{\max} \in \mathbb{R}^{\mathcal{I}\times\mathcal{O}}$. Correspondingly, we call the above-mentioned strategy as the m-PD. Then, the $\mathbf{M}^{\text{aver}}$ and $\mathbf{M}^{\max}$ can be decomposed by the SVD to measure the redundancy of filters across all layers.

With respect to SVD, a matrix $\mathbf{A}$ can be decomposed as the product of a left singular matrix $\mathbf{L}$, an eigenvalue matrix Λ, and a right singular matrix $\mathbf{R}$. The decomposition can be written as

$$\text{SVD}(\mathbf{A}) = \mathbf{L}\,\Lambda\,\mathbf{R} \tag{5}$$

where the eigenvalue matrix contains a series of sorted eigenvalues in ascending order, then we can regard each eigenvalue as the projection of the original matrix along one specified dimension (i.e., the number of eigenvalues stands for the scale of projection of $\mathbf{A}$ in low-rank space, and the absolute value of each eigenvalue stands for the strength response of projection). Reviewing the analysis of SVD at the beginning of this section, we can approximate the parameter distribution of the original matrix by analyzing the distribution of eigenvalues. It implies that we can obtain the degree of importance for a layer by the corresponding Σ. Then, we can rewrite (2) as

$$d(\Sigma) = \widetilde{N}(n^{\Sigma}, \Sigma) \tag{6}$$

where n^{Σ} stands for the number of eigenvalues and $\widetilde{N}(n^{\Sigma}, \Sigma)$ means that the shape of the distribution for the eigenvalues dominates the importance evaluation of convolutional layers.

By applying the SVD on $\mathbf{M}^{\text{aver}}$ and $\mathbf{M}^{\max}$, we can rewrite (5) as

$$\text{SVD}(\mathbf{M}^{\mathcal{I}\times\mathcal{O}}). \tag{7}$$

Subsequently, we note the eigenvalue matrices from (7) as Σ_{aver} and $\Sigma_{\max}$, and then, we can determine the importance of each convolutional layer by rewriting (6)

$$\begin{aligned} d(\Sigma_{\text{aver}}) &= \widetilde{N}(n_{\Sigma_{\text{aver}}}, \Sigma_{\text{aver}}) \\ d(\Sigma_{\max}) &= \widetilde{N}(n_{\Sigma_{\max}}, \Sigma_{\max}) \end{aligned} \tag{8}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

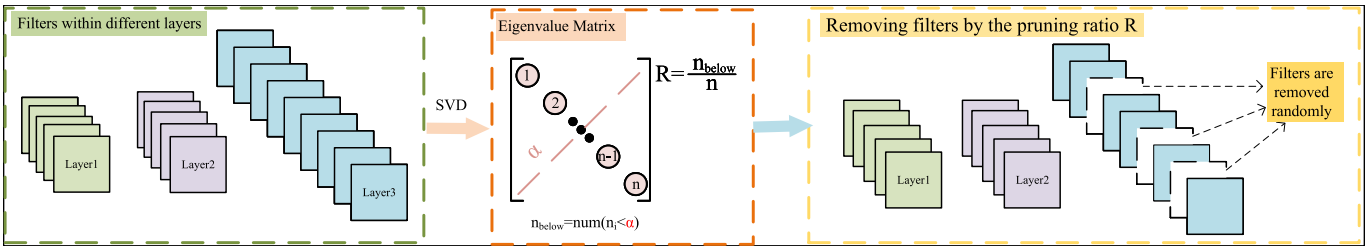


Fig. 1. Flow of the pruning framework. The circled number stands for the eigenvalue in low-rank space that is decomposed by SVD. The num() will count the number of eigenvalues that are smaller than the truncation parameter α, as well as yield n_{below}. Then, R will be used to remove filters from the corresponding layers. Since the network structure rather than parameters determines the performance of the network layer, EFPF uses a random method to reduce the number of convolution cores.

where n_{aver} and $n_{\max}$ stand for the number of eigenvalues in the low-rank space when the original tensor is folded by the average and maximum strategies, respectively. Next, we design a truncation function to calculate the energy according to the ratio of eigenvalues that are lower than a specified truncation parameter. As a result, several filters will be removed from the original layer according to the energy.

C. Calculation of Energy

In this work, we define the truncation function Θ as

$$\begin{aligned} &\Theta(d(\Sigma)) = \alpha \cap (\sigma_i) \\ &\text{s.t. } \sigma_i \in \Sigma \\ &\quad 0 < \alpha < 1 \end{aligned} \tag{9}$$

where α is the truncation parameter, and $\cap$ is the truncation operator that will cut eigenvalues if σ is less than α. Then, we try to specify the ratio of eigenvalues that should be cut by the truncation function

$$\begin{aligned} &R = \frac{n_{\text{below}}^{\Sigma}}{n^{\Sigma}} \\ &\text{s.t. } n_{\text{below}}^{\Sigma} = \text{num}(\Theta(d(\Sigma))). \end{aligned} \tag{10}$$

R will be regarded as the energy for the original convolutional layer, and then, we can remove filters by R for each layer.

In order to compare the energy of each layer at the same magnitude, we will utilize the "0–1" normalization for each eigenvalue matrix to project all the eigenvalues into the "0–1" space. The normalization of eigenvalues can assist us to calculate the energy for each layer by only using the same truncation function, and it means that we can achieve global pruning without considering the magnitude of parameters. The normalization can be written as

$$\begin{aligned} &\sigma^{(i,\text{norm})} = \frac{\sigma^i - \sigma^{\min}}{\sigma^{\max} - \sigma^{\min}} \\ &\text{s.t. } \sigma^i, \sigma^{\min}, \sigma^{\max} \in \Sigma \end{aligned} \tag{11}$$

where σ^i, $\sigma^{\min}$ and $\sigma^{\max}$ stand for the ith, the minimum and the maximum eigenvalue from an eigenvalue matrix Σ.

Reviewing the work in the previous parts, we conclude our method in Algorithm 1. $\mathcal{D}$ and f stand for the input dataset and filters, respectively. $n_{(\Sigma,l)}^{\text{below}}$ counts the number of eigenvalues that are smaller than α. It has been investigated that the structure of the network dominates the representation ability [48], and then, we can remove filters randomly from layer l while without considering the weight of each filter. After the removal of filters for each layer, we will retrain the slimmed model fully to recover the representation ability. Additionally, we will analyze the robustness of the update of parameters for EFPF in the next part.

Algorithm 1 a-PD and m-PD Pruning Method

Require: A fully trained model: $C(\mathcal{D}, f)$; The truncation parameter: α; The depth of the convolution layer: L.
Ensure: The pruned model: $C(\mathcal{D}, f')$.

```
l = 0.
while l < L do
    Calculate the eigenvalues for M_l^{aver} and M_l^{max}
    through Equation (7) with the average and maximum
    folding strategies.
    Normalize each eigenvalue by the Equation (11).
    Calculate the number of normalized eigenvalues
    n_{(Σ,l)}^{below} that is lower than the α.
    Calculate the pruning ratio of each layer R_l by the
    Equation (10).
    Remove n_{(Σ,l)}^{below} filters randomly from layer l.
    l++.
Retrain the C(D, f')
```

D. Robust Analysis for EFPF

Normally, the distribution of the parameters for a weight tensor changes rapidly at the beginning of the training process and then varies slightly in the following epochs [59]. It means that the above-mentioned EFPF can be robust to the update of the individual filter, because the energy is measured by the distribution of parameters. Then, we can get the same evaluation of layer importance even when the original model is undertrained.

This is important, because the EFPF can avoid interference from the update of parameters and thus yield consistent evaluation of layers without considering the weight information. In this part, we will explain the robustness of our method from the perspective of gradient variation.

Let us consider a 2-D matrix as $\mathbf{M}^{\mathcal{I}\times\mathcal{O}}$. Then, the eigenvalues can be obtained by the following equation:

$$|\Sigma\mathbf{E} - \mathbf{M}\mathbf{M}^T| = 0 \tag{12}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

where $\mathbf{M}^T$ is the transpose matrix of $\mathbf{M}$. $\mathbf{E}$ is the identity matrix.

Within the same convolutional layer, the updating of the parameter is flat, because they share a similar variation of the gradient with the same magnitude. We note $w^l_{(i,k)}$ as the ith parameter for the layer l when the original model is trained by k epochs, and the update of the w can be written as

$$w^l_{(i,k)} = w_{(i,k-1)} - \eta \frac{\partial \mathcal{L}_l}{\partial w^l_{(i,k-1)}} \tag{13}$$

where $w^l_{(i,k-1)}$ is the ith parameter for layer l during the $(k-1)$th training epoch, and $\mathcal{L}_{l=1}$ stands for the loss of the output for the layer $l+1$. Compared with other layers, the variation of parameters within the same layer may show higher consistency, because they are updated by the same loss function from the next layer. Therefore, we assume that the update of the parameters within the same layer is magnitude limited. Due to the learning ratio being between 0 and 1, we can approximate (13) when the gradient is pretty small

$$w^l_{(i,k)} = (1-\epsilon) w^l_{(i,k-1)}. \tag{14}$$

Consequently, we note the updated matrix as $\epsilon\mathbf{M}$, where ϵ is the variation coefficient of the gradient. Then, we can rewrite (12) as

$$\left|\Sigma^* \mathbf{E} - \left((1-\epsilon)\mathbf{M}(1-\epsilon)\mathbf{M}^T\right)\right| = 0. \tag{15}$$

Then, we can write the eigenvalue matrix for $(1-\epsilon)\mathbf{M}$ as $(1-\epsilon)^2\Sigma$. Rethinking the "0–1" normalization according to (11), we can write the normalized eigenvalues for $(1-\epsilon)^2\Sigma$ as

$$\begin{aligned}(1-\epsilon)^2\sigma^{(i,\text{norm})} &= \frac{(1-\epsilon)^2\sigma^i - (1-\epsilon)^2\sigma^{\min}}{(1-\epsilon)^2\sigma^{\max} - (1-\epsilon)^2\sigma^{\min}} \\ &= \frac{\sigma^i - \sigma^{\min}}{\sigma^{\max} - \sigma^{\min}} \\ \text{s.t. } (1-\epsilon)^2\sigma^i,\ (1-\epsilon)^2\sigma^{\min},\ (1-\epsilon)^2\sigma^{\max} &\in (1-\epsilon)^2\Sigma. \end{aligned} \tag{16}$$

Equation (16) means that we can get the similar importance evaluation of the same layer even when the original model is trained with different epochs, i.e., the model is fully trained and undertrained. Especially, when the original model is trained only with a few epochs, we can get the same evaluation as the results when the original model is trained optimally.

IV. Experiments

To validate the effectiveness of the proposed methods, we challenge the scene image classification problem on four real datasets: AID [22], PatternNet [23], NWPU45 [21], and WHU19 [24]. In this section, each of the datasets will be tested on the above-mentioned two prevalent CNN models (VGG-16 and ResNet-50) when both a-PD and m-PD are applied, and we also compare the pruning results with other NAS-based and pruning-based methods.

These experiments will be applied to demonstrate the superiority of the proposed EFPF on scene classification. At first, four real datasets and experimental settings will be described in detail. Then, the corresponding ablation experiments and contrast experiments will be introduced next. Finally, the visualization diagram is analyzed and discussed.

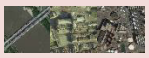
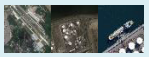
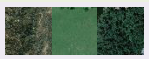
	Datasets	InputSize	TotalSize	Class
AID		600×600×3	10000	30
NWPU45		256×256×3	31500	45
PatternNet		256×256×3	30400	38
WHU19		600×600×3	1005	19

Fig. 2. Introduction of different remote sensing scene datasets. The figure mainly introduces the dataset used in the experiment and the more important indicators of the dataset: the size of the dataset, the category of the dataset, and the number of channels. The size of dataset is closely related to the size of model parameters; the classification of the dataset affects the complexity of the model, which is closely related to the accuracy of the classification results; the number of channels affects the amount of dataset required by the model.

Datasets and Settings: For all datasets, there are 80% images being grouped into the training sets, and the remaining 20% images will be regarded as the testing sets. The details of the four real datasets are shown in Fig. 2. The settings of experiment details are listed in Fig. 3. There are three learning ratios for each experiment to optimize the model, which implies that the learning ratio will change evenly for each training stage (i.e., the learning ratios will vary at 0, (1/3), and (2/3) stages, respectively). Notably, the training epochs and learning ratios will keep identical both for the baselines and the pruned models. Simultaneously, we set the truncation parameter α as 0.15, 0.20, 0.25, and 0.20, 0.25, 0.30 for VGG-16 and ResNet-50, respectively, based on extensive experiments.

AID [22] is a large aerial image dataset, which collects sample images from Google Earth images and is used to evaluate aerial images of scene classification algorithms.

PatternNet [23] is a large-scale high-resolution remote sensing dataset collected from Google Earth images or through Google Map APIs of some American cities for remote sensing image retrieval.

NWPU-RESISC45 [21] covers more than 100 countries and regions with developing, transitional, and highly developed economies in the world. It is currently a large-scale dataset. At the same time, scene images have great differences in translation, spatial resolution, viewpoint, object pose, lighting, background, and occlusion, with great intragroup differences and intergroup similarities.

WHU-RS19 [24] is a group of satellite images exported from Google Earth, including 19 types of meaningful scenes of high-resolution satellite images, which can be used for scene classification and retrieval.

A. Ablation Experiments for VGG-16

In this part, we apply our pruning framework to VGG-16 to remove redundant filters from the original model. The original models are trained with 150 and 50/20 epochs to observe whether our pruning method can maintain stable

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

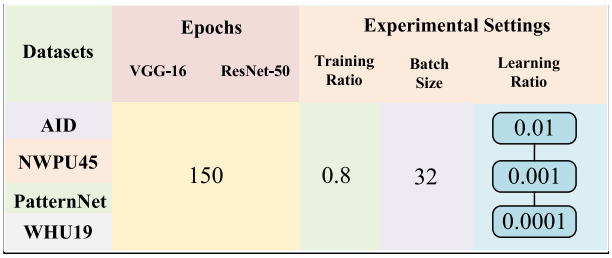


Fig. 3. Settings of experiment details. "0.01," "0.001," and "0.0001" stand for the learning ratio at every (1/3) training stage.

TABLE I

BASELINE OF VGG-16 AND RESNET-50 ON DIFFERENT DATASETS WHEN THE MODELS ARE TRAINED WITH 150 EPOCHS (FULLY TRAINING) AND 50/20 EPOCHS (UNDER TRAINING)

Models	Datasets	Param (M)	FLOPs (G)	Acc-150 (%)	Acc-50/20 (%)
VGG-16	AID	17.25	7.71	93.25	92.95
	PatternNet	17.25	7.71	97.78	95.65 (20)
	NWPU45	17.25	7.71	92.65	91.62
	WHU19	17.24	7.71	98.08	93.75
ResNet-50	AID	25.78	2.06	94.50	92.00
	PatternNet	25.78	2.06	99.54	96.02 (20)
	NWPU45	25.78	2.06	91.84	91.57
	WHU19	25.77	2.06	98.08	97.12

evaluation. The baseline results are listed in Table I, and only PatternNet is trained with 20 epochs. When the original models are trained with 150 epochs (fully training), they all present better accuracy than the performance when these models are trained with 50/20 epochs. Obviously, the ResNet-50 can yield better inference accuracy than VGG-16 on all datasets. Additionally, we also reveal the results when the above-mentioned two models are trained with 50/20 epochs (undertraining) in Table III. Compared with Table I, all the accuracy drops a bit because of the absence of efficient update of parameters and therefore cannot converge perfectly.

In order to verify the stability of our pruning framework, we apply the a-PD and m-PD to the models when they are fully trained and undertrained separately. Shown as Tables II and III, we list the performance of FLOPs, parameters, and top1 accuracy, respectively. For each dataset, we test three truncation parameters to observe the pruning results.

1) Pruning Results for the Fully Trained VGG-16: In Table II, we notice that the accuracy of all datasets outperforms the baselines when the truncation parameters (α) are set as 0.0015 and 0.0020. Especially, the accuracy can be improved by 3% for the PatternNet [23] when the truncation parameter is 0.0015. However, the accuracy fluctuated heavily when the truncation parameter is 0.0025. It seems that the removal of the filters damages the minimum topology of the network to deprive the representation ability and inevitably leads to the sacrifices of accuracy no matter how carefully the slimmed model is retrained.

However, we notice that the way the original weight tensor is folded has little impact on the accuracy of the network, as well as on the reduction of FLOPs. However, the reduction of parameters is sensitive to the folding strategies when the truncation parameters are set as 0.0020 and 0.0025, which means that the a-PD and m-PD show different reductions of parameters across all datasets. When we set the truncation parameter as 0.0015, the remaining parameters and FLOPs approach 14.89M and 7.71G both for a-PD and m-PD. It means that there are 13.62% parameters are pruned without the reduction of FLOPs. Therefore, 0.0015 is a poor initialization to prune the fully trained VGG-16 on different remote sensing datasets.

When we revise α to 0.0020, the remaining FLOPs for a-PD and m-PD methods are near 5.60M and 5.30M, respectively. It implies that there are 27.30% and 31.25% reductions, respectively. However, the remaining parameters are around 5.35M and 4.00M for a-PD and m-PD, respectively. Thus, 64.10% and 73.15% reduction is confirmed for all datasets.

Moreover, when we continue to improve α to 0.0025, there are more reductions of FLOPs and parameters but cannot instruct the slimmed model to converge through the gradient descent. Therefore, we claim that 0.0025 is also a trivial initialization of α to prune the VGG-16.

2) Pruning Results for the Undertrained VGG-16: Surprisingly, the performance of the pruning results for the undertrained VGG-16 also shows similarity to the fully trained VGG-16, including the accuracy, FLOPs, and parameters.

When α is denoted as 0.0015, the accuracy of all slimmed models outperforms the baselines obviously, as well as shows the same representation ability as the fully trained VGG-16. Additionally, the FLOPs and parameters can also keep extremely similar to the results in Table II. Then, we set α as 0.0020 and 0.0025 to observe the variation of the pruning results on the undertrained models. Just as expected, all the results show a similarity to the fully trained VGG-16. That is, 0.0020 is an appropriate initialization for α to implement our methods on the VGG-16. The FLOPs and parameters can not only be reduced by 30% and 73% around maximally but also improve the top1 accuracy to some extent. While the results have degenerated obviously when α is set as 0.0025, and accordingly, the slimmed models cannot be used to classify the remote sensing scene images efficiently.

By the analysis of extensive experiments, we verify that EFPF is efficient to observe the redundancy among filters, and it can maintain stability across different trained models, i.e., the pruning results show strong similarity both for the above-mentioned fully trained and undertrained models. Moreover, we explore the effect of truncation parameters and then yield impressive results with the value of 0.0020. In the following parts, we extend the EFPF on ResNet-50 to probe the performance.

B. Ablation Experiments for ResNet-50

The ResNet-50 is also constructed by the stack of convolutional layers while with the skip connections across different blocks, and therefore, can achieve better top1 accuracy than

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

TABLE II

PRUNING RESULTS OF THE FULLY TRAINED VGG-16 ON DIFFERENT DATASETS WHEN α IS SET AS 0.0015, 0.0020, AND 0.0025, RESPECTIVELY. THE "ACCU-O" STANDS FOR THE ORIGINAL TOP1 ACCURACY

Models (Epochs)	Datasets	Trunc-Param	FLOPs(a-PD) (G)	FLOPs(m-PD) (G)	Par(a-PD) (M)	Par(m-PD) (M)	Accu(a-PD) (%)	Accu(m-PD) (%)	Accu-O (%)
VGG-16(150)	AID	0.0015	7.71	7.71	14.88	14.88	94.95	94.02	93.25
		0.0020	5.63	5.33	5.32	3.98	94.55	94.02	
		0.0025	5.23	5.15	3.12	2.94	84.80	51.55	
	PatternNet	0.0015	7.71	7.71	14.89	14.89	99.74	99.67	97.78
		0.0020	5.64	5.33	5.36	4.00	99.80	99.59	
		0.0025	5.23	5.16	3.12	2.94	97.07	81.76	
	NWPU45	0.0015	7.71	7.71	14.89	14.89	93.54	92.97	92.65
		0.0020	5.51	5.31	4.63	3.77	93.65	92.67	
		0.0025	5.21	5.16	3.07	2.94	64.29	27.79	
	WHU19	0.0015	7.71	7.71	14.88	14.88	99.04	98.08	98.08
		0.0020	5.64	5.32	3.38	3.98	99.04	98.08	
		0.0025	5.22	5.16	3.10	2.94	47.60	32.69	

TABLE III

PRUNING RESULTS OF THE UNDERTRAINED VGG-16 ON DIFFERENT DATASETS WHEN α IS SET AS 0.0015, 0.0020, AND 0.0025, RESPECTIVELY. THE "ACCU-O" STANDS FOR THE ORIGINAL TOP1 ACCURACY

Models (Epochs)	Datasets	Trunc-Param	FLOPs(a-PD) (G)	FLOPs(m-PD) (G)	Par(a-PD) (M)	Par(m-PD) (M)	Accu(a-PD) (%)	Accu(m-PD) (%)	Accu-O (%)
VGG-16(50/20)	AID	0.0015	7.71	7.71	14.88	14.89	93.15	94.35	92.95
		0.0020	5.62	5.33	5.23	3.99	94.34	93.90	
		0.0025	5.23	5.16	3.12	2.94	75.10	40.90	
	PatternNet (20)	0.0015	7.70	7.70	14.78	14.79	98.55	98.15	95.65
		0.0020	5.61	5.31	5.29	3.93	94.45	94.01	
		0.0025	5.49	5.29	4.60	3.72	93.35	92.57	
	NWPU45	0.0015	7.71	7.71	14.89	14.89	92.16	92.02	91.62
		0.0020	5.54	5.32	4.74	3.82	92.62	92.21	
		0.0025	5.22	5.16	3.11	2.94	59.71	2.22	
	WHU19	0.0015	7.71	7.71	14.88	14.88	95.67	95.67	93.75
		0.0020	5.64	5.32	5.39	3.98	95.19	91.83	
		0.0025	5.22	5.16	3.10	2.94	43.75	12.02	

the VGG-16. The baselines of the fully trained and the undertrained models are shown in Tables IV and V. Obviously, for the same dataset, ResNet-50 shows better test top1 accuracy while with the larger occupation of parameters. Therefore, the reduction of parameters is necessary to improve the efficiency of the ResNet-50 on the remote sensing datasets.

Similar to the VGG-16, the undertrained models show relatively lower accuracy than the results when the models are trained fully. Consequently, we test our pruning framework both when the original models are trained with 150 and 50/20 epochs, respectively.

Notably, there are three kinds of filters that have 1×1, 3×3, and 7×7 sizes, which contribute to distinct representation ability for the whole network. Due to the dimensional constraints of size, we only remove the 3×3 filters. Then, parameters and FLOPs can be downsized apparently.

1) Pruning Results for the Fully Trained ResNet-50: The pruning results are displayed in Table IV. All the slimmed models can clearly yield better accuracy than the baselines when the truncation parameter α is set as 0.0015, 0.0020, and 0.0025. However, no matter how we fold the original weight tensor, the accuracy is similar both for the a-PD and m-PD methods. For example, there are almost 3% improvements in accuracy for the AID [22] and PatternNet [23] when the original models are pruned by different α. The 4% and 1% improvements in the accuracy can be achieved for the NWPU45 [21] and WHU19 [24] datasets, respectively. Even when α is set as 0.0025, the slimmed models can recover the original accuracy, which means that the ResNet-50 is more robust than VGG-16 when the structure of the models is damaged.

The ResNet-50 can also show strong robustness in the reduction of parameters and FLOPs. It means that the parameters and FLOPS can stay identical for all datasets when α and the folding methods are the same. For example, with respect to the a-PD method, there are 0.49%, 7.77%, and 11.17% reductions of FLOPs when α is set as 0.0015, 0.0020, and 0.0025, respectively. The FLOPs of the m-PD method are also similar to the a-PD method. For the parameters, there are 1.82%, 24.55%, and 36.23% reductions for the a-PD method, and there are 0, 30.26%, and 39.60% reductions for the m-PD method.

Obviously, the initialization of 0.0025 for α is efficient to yield considerable reductions for parameters and FLOPs and achieve better top1 accuracy than the baselines.

2) Pruning Results for the Undertrained ResNet-50: The pruned results are shown in Table V. Compared with the results in Table IV, we observe that all the parameters and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

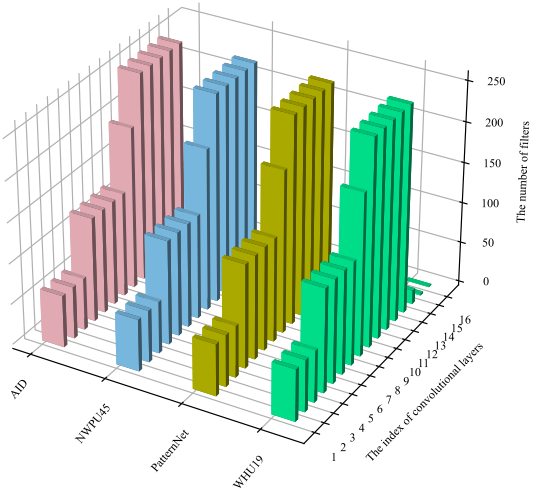


Fig. 4. Structure of the slimmed ResNet-50 when we set α as 0.0030 for AID, NWPU45, PatternNet, and WHU19, respectively.

FLOPs show identical reductions as the results when the original models are trained fully. The accuracy can also outperform the baselines but with little sacrifice for the results in Table IV. In conclusion, 0.0025 is an efficient initialization for α to remove redundant filters across all layers for the ResNet-50. However, our pruning framework can also show strong consistency of filters' removal for the ResNet-50.

C. Comparisons With NAS and Other Pruning Methods

In this section, we compare EFPF with prevalent NAS-based methods and pruning methods. Notably, the ResNet-50 performs better than the VGG-16 in accuracy, and therefore, we will compare our methods with others on the ResNet-50. It is obvious that the slimmed ResNet-50 can also show strong representation ability even when α is 0.0025; then, we continue to increase α to 0.0030 to observe the variation of accuracy, FLOPs, and parameters. We test the experiments for all datasets on the ResNet-50, and the outcomes are listed in Table VI. Simultaneously, we also list the structure of the pruned model in Fig. 4. It shows that the structures of the slimmed model are similar, and deep layers tend to be pruned heavily.

For the NAS-based models, we select three methods that have achieved impressive results on NWPU45 [21]. The "Random Search" [60] method achieves the best performance among all the mentioned methods; however, all the NAS-based methods are time-costing during the search process. It means that we need to prepare massive hardware to run the NAS methods which then leads to extra consumption of computation resources. Simultaneously, we apply "Random Search" [60] and "Gradient-based" [61] methods for the AID [22] datasets. The "Random Search" [60] can also perform higher accuracy than the "Gradient-based" [61] method but inevitably lead to heavier searching costs. For PatternNet [23] and WHU19 [24], we focus on pruning methods, because the size of these datasets is relatively small. It means that similar performance tends to be yielded for all methods, and the pruning techniques can save searching time greatly.

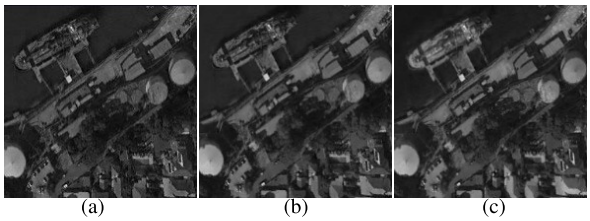


Fig. 5. Three channels of one input "Ship" image that is obtained from the NWPU45 dataset. (a) Channel 1. (b) Channel 2. (c) Channel 3.

We introduce two global pruning methods that are used to remove redundant filters from all the layers by "Taylor Pruning" [15] and "Network Slimming" [16], respectively. It should be pointed out that the pruning methods are searching-efficient, which can finish the removal of filters in only a few hours with one 2080Ti. When we set α as 0.0030, we count the number of pruned filters and calculate the overall pruning ratio as **41.66**%. Then, we specify both the overall pruning ratio of "Taylor Pruning" and "Network Slimming" [16] as **41.66**%. Clearly, all three pruning methods can achieve the improvement of top1 accuracy for all datasets. It can be explained that removing the filters from convolutional layers can work as the "Dropout" techniques, which breaks up the constraints of overfitting and then improve the representation ability. Especially, EFPF performs the best in the improvement of the top1 accuracy among these three pruning methods for all datasets, and we also achieve the best reduction of parameters. For the FLOPs, our method is better than the "Network Slimming" [16] while is a bit worse than "Taylor Pruning" [15] for all the datasets. In summary, the searching cost of EFPF is lower than the NAS-based method, and we can also achieve comparable results as the SceneNet [62] and RSNet [63]. In addition, our pruning method outperforms the prevalent "Taylor Pruning" [15] and "Network Slimming" [16] in accuracy, and the reduction of parameters are the highest in these three pruning methods. The reductions of FLOPs are also competitive compared with the other two pruning methods with the same magnitude of searching consumption.

D. Efficiency Analysis by the Visualization of Features

The representation ability of each convolutional layer can be illustrated by the output features, and therefore, we aim for revealing the distinction between the original and the pruned models, as well as between the fully trained (150 epochs) and undertrained (50/20 epochs) models by visualizing the features. Furthermore, if the colors are deeper, the absolute values for the corresponding parameters are larger. Because the structure of VGG-16 is simpler than ResNet-50, we will illustrate the efficacy of our method by visualizing the features of VGG-16. Notably, there are 13 convolutional layers for VGG-16, and we rename them "layer1–layer13" or "l1–l13." Specifically, we show three layers with "layer1," "layer8," and "layer13" from shallow to deep structures, respectively. Then, the visualizations of features for the VGG-16 on the NPWPU45 are displayed from Figs. 6–11 in this part.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

PRUNING RESULTS OF THE FULLY TRAINED RESNET-50 ON DIFFERENT DATASETS WHEN α IS SET AS 0.0015, 0.0020, AND 0.0025, RESPECTIVELY. THE "ACCU-O" STANDS FOR THE ORIGINAL TOP1 ACCURACY

Models (Epochs)	Datasets	Trunc-Param	FLOPs(a-PD) (G)	FLOPs(m-PD) (G)	Par(a-PD) (M)	Par(m-PD) (M)	Accu(a-PD) (%)	Accu(m-PD) (%)	Accu-O (%)
ResNet-50(150)	AID	0.0015	2.05	2.05	25.32	25.78	93.15	94.35	94.50
		0.0020	1.90	1.87	19.44	17.98	96.55	96.90	
		0.0025	1.83	1.81	16.45	15.57	97.20	96.65	
	PatternNet	0.0015	2.05	2.05	25.31	25.78	99.87	99.90	99.54
		0.0020	1.90	1.87	19.44	17.98	99.84	99.47	
		0.0025	1.83	1.81	16.45	15.57	99.92	99.90	
	NWPU45	0.0015	2.05	2.05	25.31	25.78	95.78	95.65	91.84
		0.0020	1.90	1.87	19.44	17.99	95.81	95.02	
		0.0025	1.83	1.81	16.44	15.57	95.51	95.46	
	WHU19	0.0015	2.05	2.05	25.31	25.77	98.56	99.04	98.08
		0.0020	1.90	1.87	19.46	17.98	99.52	99.90	
		0.0025	1.83	1.81	16.44	15.57	100.00	98.56	

TABLE V

PRUNING RESULTS OF THE UNDERTRAINED RESNET-50 ON DIFFERENT DATASETS WHEN α IS SET AS 0.0015, 0.0020, AND 0.0025, RESPECTIVELY. THE "ACCU-O" STANDS FOR THE ORIGINAL TOP1 ACCURACY

Models (Epochs)	Datasets	Trunc-Param	FLOPs(a-PD) (G)	FLOPs(m-PD) (G)	Par(a-PD) (M)	Par(m-PD) (M)	Accu(a-PD) (%)	Accu(m-PD) (%)	Accu-O (%)
ResNet-50(50/20)	AID	0.0015	2.05	2.05	25.34	25.78	96.405	96.75	92.00
		0.0020	1.90	1.87	19.42	17.96	95.85	96.10	
		0.0025	1.83	1.81	16.44	16.44	96.25	95.60	
	PatternNet (20)	0.0015	2.04	2.02	25.23	25.68	99.72	99.74	96.02
		0.0020	1.89	1.83	19.26	17.78	99.72	99.06	
		0.0025	1.81	1.79	16.24	16.24	99.65	99.62	
	NWPU45	0.0015	2.05	2.05	25.33	25.78	94.91	95.19	91.57
		0.0020	1.90	1.87	19.44	17.99	94.90	94.14	
		0.0025	1.83	1.81	16.44	16.44	95.46	94.49	
	WHU19	0.0015	2.05	2.05	25.31	25.77	98.56	99.04	97.12
		0.0020	1.90	1.87	19.46	17.98	99.04	98.08	
		0.0025	1.83	1.81	16.45	16.45	98.56	97.60	

TABLE VI

COMPARISONS OF DIFFERENT COMPRESSION TECHNIQUES WITH OUR METHOD (EFPF). THE "GDAYS" MEANS THE TIME COSTS OF THE CORRESPONDING METHODS WITH ONE 2080TI

Dataset	Method		Top1-Accuracy	FLOPs	Parameters	Searching Cost (GDays)
AID	NAS-Based	Random Search [60]	96.10%	0.56G	5.10M	9.6
		Gradient-Based [61]	91.12%	-	3.80M	-
	Pruning-Based	Taylor Pruning [15]	96.15%	1.66G	18.13M	0.13
		Network Slimming [16]	96.15%	1.81G	18.02M	0.15
		EFPF	**97.00%**	**1.79G**	**15.49M**	**0.13**
PatternNet	Pruning-Based	Taylor Pruning [15]	99.80%	1.47G	18.33M	0.12
		Network Slimming [16]	99.85%	1.86G	17.42M	0.13
		EFPF	**99.85%**	**1.79G**	**15.49M**	**0.12**
NWPU45	NAS-Based	SceneNet [62]	95.22%	9.47G	1.02M	10
		RSNet [63]	92.66%	1.28G	1.40M	2
		Random Search [60]	96.56%	0.56G	5.10M	9.6
	Pruning-Based	Taylor Pruning [15]	94.80%	1.59G	18.29M	0.13
		Network Slimming [16]	94.53%	1.93G	17.66M	0.15
		EFPF	**95.43%**	**1.79G**	**15.49M**	**0.13**
WHU19	Pruning-Based	Taylor Pruning [15]	98.73%	1.71G	17.78M	0.05
		Network Slimming [16]	98.91%	1.88G	16.97M	0.05
		EFPF	**99.04%**	**1.79G**	**15.49M**	**0.05**

1) Features of the Baselines: We pick up one "ship" picture from the NWPU45 and display it in Fig. 5. There are three channels for the image, and then we feed this picture into different models to observe the variation of the features. The

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

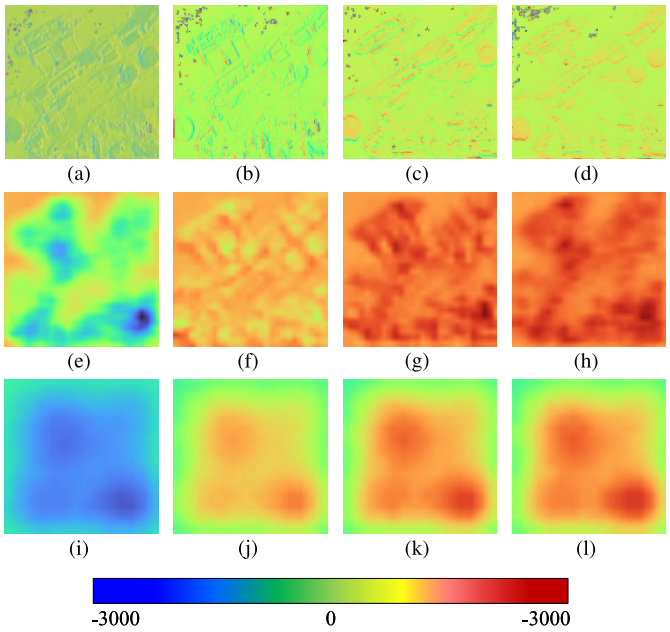


Fig. 6. Features of layer1, layer8, and layer13 for the VGG-16 that has been trained with 150 epochs. We pick up four features for each layer to show the representative details. (a) 150-layer1-1. (b) 150-layer1-2. (c) 150-layer1-3. (d) 150-layer1-4. (e) 150-layer8-1. (f) 150-layer8-2. (g) 150-layer8-3. (h) 150-layer8-4. (i) 150-layer13-1. (j) 150-layer13-2. (k) 150-layer13-3. (l) 150-layer13-4.

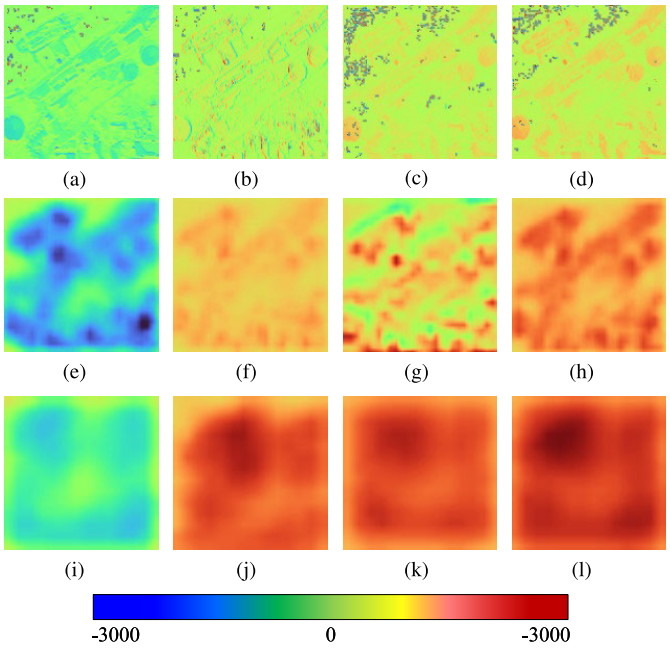


Fig. 7. Features of layer1, layer8, and layer13 for the VGG-16 that has been trained with 50 epochs. We pick up four features for each layer to show the representative details. (a) 50-layer1-1. (b) 50-layer1-2. (c) 50-layer1-3. (d) 50-layer1-4. (e) 50-layer8-1. (f) 50-layer8-2. (g) 50-layer8-3. (h) 50-layer8-4. (i) 50-layer13-1. (j) 50-layer13-2. (k) 50-layer13-3. (l) 50-layer13-4.

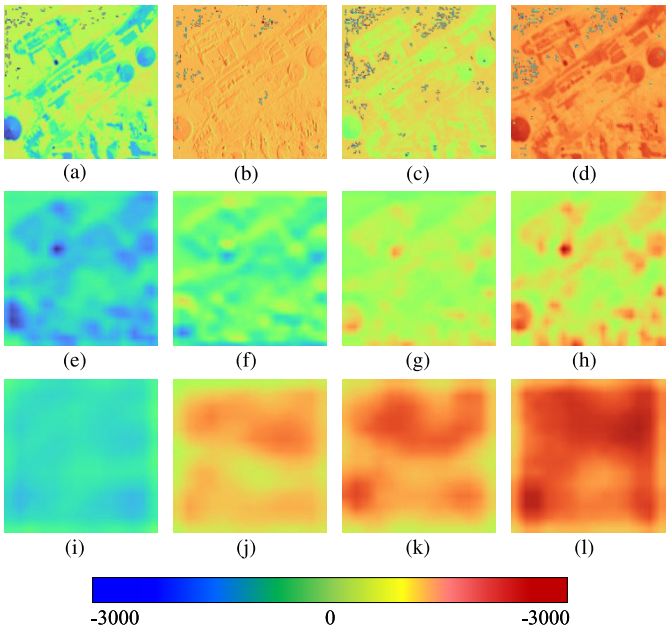


Fig. 8. Features of layer1, layer8, and layer13 for the pruned model. The 150 means that the original model is trained with 150 epochs, and the "aver" means that the original weight tensor is folded with the a-PD strategy. (a) P-150-l1-1-a. (b) P-150-l1-2-a. (c) P-150-l1-3-a. (d) P-150-l1-4-a. (e) P-150-l8-1-a. (f) P-150-l8-2-a. (g) P-150-l8-3-a. (h) P-150-l8-4-a. (i) P-150-l13-1-a. (j) P-150-l13-2-a. (k) P-150-l13-3-a. (l) P-150-l13-4-a.

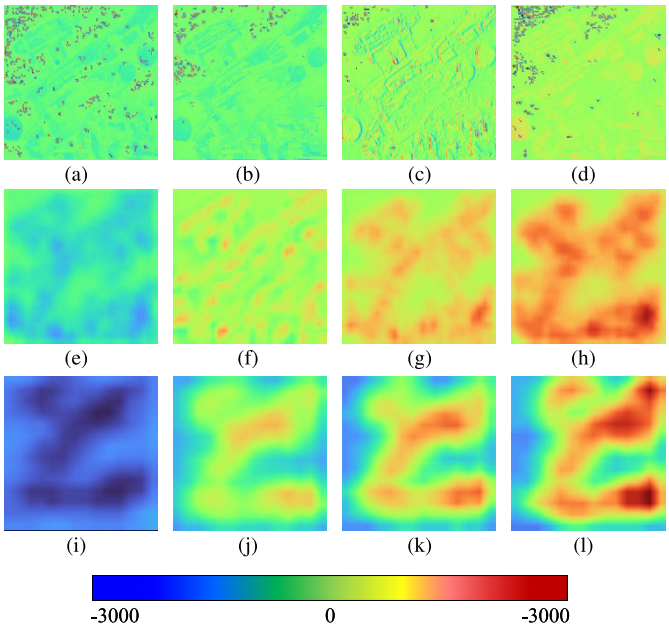


Fig. 9. Features of layer1, layer8, and layer13 for the pruned model. The 150 means that the original model is trained with 150 epochs, and the "max" means that the original weight tensor is folded with the m-PD strategy. (a) P-150-l1-1-m. (b) P-150-l1-2-m. (c) P-150-l1-3-m. (d) P-150-l1-4-m. (e) P-150-l8-1-m. (f) P-150-l8-2-m. (g) P-150-l8-3-m. (h) P-150-l8-4-m. (i) P-150-l13-1-m. (j) P-150-l13-2-m. (k) P-150-l13-3-m. (l) P-150-l13-4-m.

baselines are shown in Figs. 6 and 7. Notably, the degree of colors stands for the different values. The blue color means that the value is negative, and the red color means that the value is positive. For the low-level features, there are only tiny losses of shapes and textures. The middle-level and high-level features are abstract as the features in Figs. 8 and 9, which means that the low-level features have the priority to contribute to the representation abilities. We observe that the shallow layer tends to capture the low-level features such as the texture and the contour. The deep layer shows high-level features that are hard to discern. The features of the middle layer combine

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

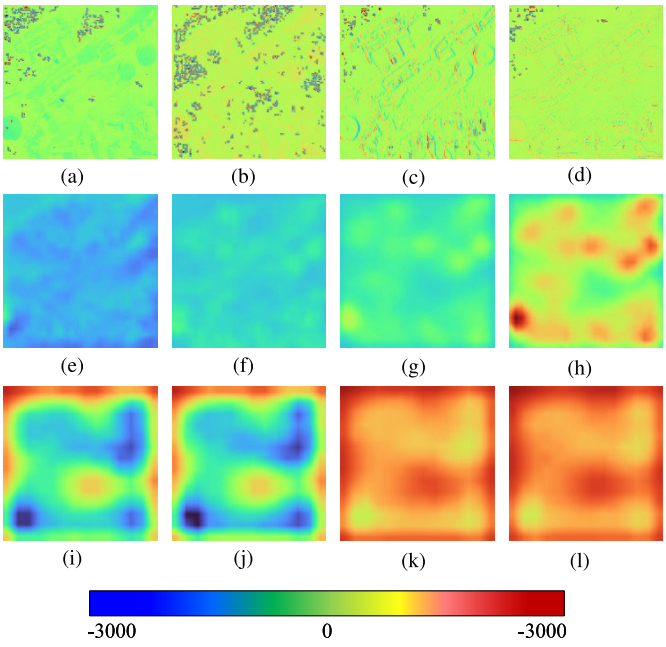


Fig. 10. Features of layer1, layer8, and layer13 for the pruned model. The 50 means that the original model is trained with 50 epochs, and the "aver" means that the original weight tensor is folded with the a-PD strategy. (a) P-50-l1-1-a. (b) P-50-l1-2-a. (c) P-50-l1-3-a. (d) P-50-l1-4-a. (e) P-50-l8-1-a. (f) P-50-l8-2-a. (g) P-50-l8-3-a. (h) P-50-l8-4-a. (i) P-50-l13-4-a. (j) P-50-l13-2-a. (k) P-50-l13-3-a. (l) P-50-l13-4-a.

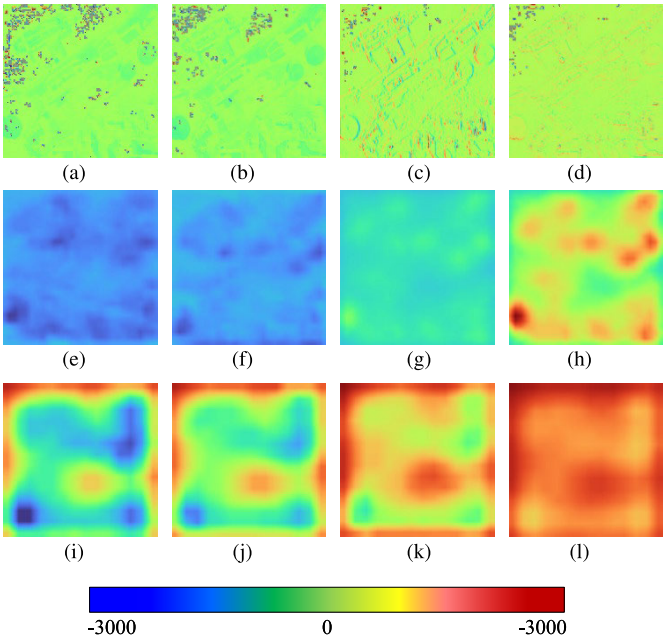


Fig. 11. Features of layer1, layer8, and layer13 for the pruned model. The 50 means that the original model is trained with 50 epochs, and the "max" means that the original weight tensor is folded with the m-PD strategy. (a) P-50-l1-1-m. (b) P-50-l1-2-m. (c) P-50-l1-3-m. (d) P-50-l1-4-m. (e) P-50-l8-1-m. (f) P-50-l8-2-m. (g) P-50-l8-3-m. (h) P-50-l8-4-m. (i) P-50-l13-1-m. (j) P-50-l13-2-m. (k) P-50-l13-3-m. (l) P-50-l13-4-m.

the attributes for both the shallow and deep layers, which show the obscure shape of the ship.

With respect to the different baselines, we observe that the undertrained model can capture the majority of attributes as the fully trained model. For example, the different levels of features appear from shallow to deep layers. However, the undertrained model seems to be polluted by noise, i.e., more spots are founded in Fig. 7(a)–(d), and the middle-level features in Fig. 7(e)–(h) are more vague than the features in Fig. 6(e)–(h).

For the high-level features, the difference between the two models is narrow, which means that the deep layers are easier to converge. Consequently, the fully trained model can show better performance than the undertrained models. It can also illustrate that there may be more redundancy in the deep layers, because they are robust to the update of parameters.

2) Features of the Slimmed Models: We display the features of different slimmed models when we set the truncation parameter α as 0.0020. Furthermore, we show the results of different folding strategies in Figs. 8–11 both for the fully trained and for undertrained models.

For the fully trained models, the features of "layer1" for both a-PD and m-PD strategies can show more clear low-lever attributes. Especially, there is less noise in Fig. 8(a)–(d), and it implies that the corresponding slimmed model may show better representation ability than the model that is pruned with the m-PD strategy. The middle-level and high-level features are similar, and they are hard to distinguish by straight observation. In general, reviewing the pruned results in Fig. 4, we can claim that the low-level features dominate the representation ability of the models, namely, redundancy appears to exist in deep layers.

In light of the illustration of Figs. 10 and 11, we observe that the shallow layers can still show clear contours as the undertrained baselines, and the noise is also similar to Fig. 7.

In summary, the above-mentioned visualization of features reinforces the concept that there is more redundancy in deep layers, and the EFPF can work well for both the fully trained and undertrained models.

V. Conclusion

The method proposed in this article is proven to be effective through experiments. By leveraging the eigenvalues of each convolutional layer via SVD, EFPF can determine the importance of each convolution for CNNs on remote sensing scene images at the early training stage. Then, we can remove filters proportionally through a pre-defined truncation parameter. It is verified that our algorithm can work stably even when the original model is trained poorly. It implies that our method can prune the original model at the early training stage and then saves computation resources on the satellite platform. In the experimental results, EFPF can not only ensure the accuracy of the classification results of the scene classification but also achieve a higher pruning rate, making the pruning effect reach a better level. In the experiment, the proposed framework can cut the FLOPs and parameters obviously while improving the top1 accuracy slightly.

References

[1] J. Leitloff, S. Hinz, and U. Stilla, "Vehicle detection in very high resolution satellite images of city areas," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 7, pp. 2795–2806, Jul. 2010.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

[2] N. Longbotham, C. Chaapel, L. Bleiler, C. Padwick, W. J. Emery, and F. Pacifici, "Very high resolution multiangle urban classification analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 4, pp. 1155–1170, Apr. 2012.

[3] Y. Lv, Y. Duan, W. Kang, Z. Li, and F.-Y. Wang, "Traffic flow prediction with big data: A deep learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 865–873, Apr. 2015.

[4] H. Sun, S. Li, X. Zheng, and X. Lu, "Remote sensing scene classification by gated bidirectional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 82–96, Jan. 2020.

[5] N. He, L. Fang, S. Li, J. Plaza, and A. Plaza, "Skip-connected covariance network for remote sensing scene classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 5, pp. 1461–1474, May 2020.

[6] T. Liu et al., "Building change detection for VHR remote sensing images via local–global pyramid network and cross-task transfer learning strategy," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4704817.

[7] J. Li, M. Gong, H. Liu, Y. Zhang, M. Zhang, and Y. Wu, "Multiform ensemble self-supervised learning for few-shot remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4500416.

[8] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Aging evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 5048–5056.

[9] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–16.

[10] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the human out of the loop: A review of Bayesian optimization," *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, Jan. 2016, doi: 10.1109/JPROC.2015.2494218.

[11] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jun. 2018, pp. 1–12.

[12] Y. LeCun, J. Denker, and S. Solla, "Optimal brain damage," in *Proc. Adv. Neural Inf. Process. Syst.*, Denver, CO, USA, Jan. 1990, pp. 598–605.

[13] B. Hassibi, D. Stork, G. Wolff, and T. Watanabe, "Optimal brain surgeon," in *Proc. Adv. Neural Inf. Process. Syst.*, Denver, CO, USA, Nov. 1993, pp. 263–270.

[14] D. Molchanov, A. Ashukha, and D. Vetrov, "Variational dropout sparsifies deep neural networks," in *Proc. Int. Conf. Mach. Learn.*, Sydney, NSW, Australia, Aug. 2017, pp. 2498–2507.

[15] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," 2016, *arXiv:1611.06440*.

[16] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 2755–2763.

[17] J.-H. Luo, J. Wu, and W. Lin, "ThiNet: A filter level pruning method for deep neural network compression," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 5068–5076.

[18] C. Tai, T. Xiao, Y. Zhang, X. Wang, and E. Weinan, "Convolutional neural networks with low-rank regularization," in *Proc. Int. Conf. Learn. Represent.*, 2016, pp. 1–11.

[19] X. Zhang, J. Zou, K. He, and J. Sun, "Accelerating very deep convolutional networks for classification and detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 10, pp. 1943–1955, Oct. 2016, doi: 10.1109/TPAMI.2015.2502579.

[20] T. Huang, R. Zhao, L. Bi, D. Zhang, and C. Lu, "Neural embedding singular value decomposition for collaborative filtering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 6021–6029, Oct. 2022, doi: 10.1109/TNNLS.2021.3070853.

[21] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.

[22] G. Xia et al., "AID: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, Jul. 2017.

[23] W. Zhou, S. Newsam, C. Li, and Z. Shao, "PatternNet: A benchmark dataset for performance evaluation of remote sensing image retrieval," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 197–209, Nov. 2018.

[24] D. Dai and W. Yang, "Satellite image classification via two-layer sparse coding with biased image representation," *IEEE Geosci. Remote Sens. Lett.*, vol. 8, no. 1, pp. 173–176, Jan. 2011.

[25] L. Gómez-Chova, D. Tuia, G. Moser, and G. Camps-Valls, "Multimodal classification of remote sensing images: A review and future directions," *Proc. IEEE*, vol. 103, no. 9, pp. 1560–1584, Sep. 2015.

[26] D. Tuia, M. Volpi, L. Copa, M. Kanevski, and J. Munoz-Mari, "A survey of active learning algorithms for supervised remote sensing image classification," *IEEE J. Sel. Topics Signal Process.*, vol. 5, no. 3, pp. 606–617, Jun. 2011.

[27] P. Ghamisi, J. Plaza, Y. Chen, J. Li, and A. J. Plaza, "Advanced spectral classifiers for hyperspectral images: A review," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 1, pp. 8–32, Mar. 2017.

[28] L. He, J. Li, C. Liu, and S. Li, "Recent advances on spectral–spatial hyperspectral image classification: An overview and new guidelines," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 3, pp. 1579–1597, Mar. 2018.

[29] S. Li, W. Song, L. Fang, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Deep learning for hyperspectral image classification: An overview," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6690–6709, Sep. 2019.

[30] L. Ma, Y. Liu, X. Zhang, Y. Ye, G. Yin, and B. A. Johnson, "Deep learning in remote sensing applications: A meta-analysis and review," *ISPRS J. Photogramm. Remote Sens.*, vol. 152, pp. 166–177, Jun. 2019.

[31] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, Lake Tahoe, NV, USA, Dec. 2012, pp. 1106–1114.

[32] K. Simonyan and A. Zissermann, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent.*, San Diego, CA, USA, May 2015, pp. 1646–1654.

[33] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.

[34] X. Liu, Y. Zhou, J. Zhao, R. Yao, B. Liu, and Y. Zheng, "Siamese convolutional neural networks for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 8, pp. 1200–1204, Aug. 2019.

[35] R. Zhu, L. Yan, N. Mo, and Y. Liu, "Attention based deep feature fusion for the scene classification of high resolution remote sensing images," *Remote Sens.*, vol. 11, no. 17, p. 1996, Aug. 2019.

[36] M. Gong, J. Li, Y. Zhang, Y. Wu, and M. Zhang, "Two-path aggregation attention network with quad-patch data augmentation for few-shot scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4511616.

[37] F. Jiang, M. Gong, H. Zheng, T. Liu, M. Zhang, and J. Liu, "Self-supervised global–local contrastive learning for fine-grained change detection in VHR images," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4400613.

[38] W. Zhang, P. Tang, and L. Zhao, "Remote sensing image scene classification using CNN-CapsNet," *Remote Sens.*, vol. 11, no. 5, p. 494, Feb. 2019.

[39] R. Minetto, M. Pamplona Segundo, and S. Sarkar, "Hydra: An ensemble of convolutional neural networks for geospatial land classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6530–6541, Sep. 2019.

[40] S. Chaib, H. Liu, Y. Gu, and H. Yao, "Deep feature fusion for VHR remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4775–4784, Aug. 2017.

[41] N. He, L. Fang, S. Li, A. Plaza, and J. Plaza, "Remote sensing scene classification using multilayer stacked covariance pooling," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 12, pp. 6899–6910, Dec. 2018.

[42] X. Lu, H. Sun, and X. Zheng, "A feature aggregation convolutional neural network for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 7894–7906, Oct. 2019.

[43] F. Zhang, B. Du, and L. Zhang, "Scene classification via a gradient boosting random convolutional network framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 3, pp. 1793–1802, Mar. 2016.

[44] Y. Zhong, F. Fei, and L. Zhang, "Large patch convolutional neural networks for the scene classification of high spatial resolution imagery," *J. Appl. Remote Sens.*, vol. 10, no. 2, Apr. 2016, Art. no. 025006.

[45] Y. Liu, Y. Zhong, and Q. Qin, "Scene classification based on multiscale convolutional neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 12, pp. 7109–7121, Dec. 2018.

[46] J. Fang, Y. Yuan, X. Lu, and Y. Feng, "Robust space–frequency joint representation for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 7492–7502, Oct. 2019.

[47] J. Xie, N. He, L. Fang, and A. Plaza, "Scale-free convolutional neural network for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6916–6928, Sep. 2019.

[48] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–42.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.

[49] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. Graf, "Pruning filters for efficient ConvNets," in *Proc. Int. Conf. Learn. Represent.*, Toulon, France, Apr. 2017.
[50] J. Ye et al., "Rethinking the smaller-norm-less-informative assumption in channel pruning of convolution layers," in *Proc. Int. Conf. Learn. Represent.*, Apr./May 2018, pp. 1–11.
[51] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 1398–1406.
[52] C. Zheng, J. Wang, A. Ma, and Y. Zhong, "AutoLC: Search lightweight and top-performing architecture for remote sensing image land-cover classification," 2022, *arXiv:2205.05369*.
[53] W. Jing, Q. Ren, J. Zhou, and H. Song, "AutoRSISC: Automatic design of neural architecture for remote sensing image scene classification," *Pattern Recognit. Lett.*, vol. 140, pp. 186–192, Dec. 2020.
[54] C. Peng, Y. Li, L. Jiao, and R. Shang, "Efficient convolutional neural architecture search for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 6092–6105, Jul. 2021.
[55] J. Chen et al., "Contextual information-preserved architecture learning for remote-sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5602614.
[56] Z. Zhang, S. Liu, Y. Zhang, and W. Chen, "RS-DARTS: A convolutional neural architecture search for remote sensing image scene classification," *Remote Sens.*, vol. 14, no. 1, p. 141, Dec. 2021.
[57] Z. Liu, M. Sun, T. Zhou, G. Huang, and D. Trevor, "Rethinking the value of network pruning," in *Proc. Int. Conf. Learn. Represent.*, HI, Jan. 2019, pp. 1–11.
[58] Z. Huang, W. Shao, X. Wang, L. Lin, and P. Luo, "Rethinking the pruning criteria for convolutional neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2021, pp. 1–14.
[59] J. Frankle, D. J. Schwab, and A. S. Morcos, "The early phase of neural network training," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–20.
[60] C. Broni-Bediako, Y. Murata, L. H. B. Mormille, and M. Atsumi, "Searching for CNN architectures for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4701813.
[61] J. Chen et al., "Convolution neural network architecture learning for remote sensing scene classification," 2020, *arXiv:2001.09614*.
[62] A. Ma, Y. Wan, Y. Zhong, J. Wang, and L. Zhang, "SceneNet: Remote sensing scene classification deep learning network using multi-objective neural evolution architecture search," *ISPRS J. Photogramm. Remote Sens.*, vol. 172, pp. 171–188, Feb. 2021.
[63] J. Wang, Y. Zhong, Z. Zheng, A. Ma, and L. Zhang, "RSNet: The search for remote sensing deep neural networks in recognition tasks," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2520–2534, Mar. 2021.



Yiheng Lu received the B.Eng. degree from Shantou University, Shantou, China, in 2018. He is currently pursuing the Ph.D. degree with Xidian University, Xi'an, China.

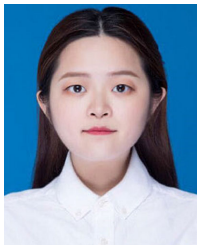
His research interests include deep learning and structure optimization.



Maoguo Gong (Senior Member, IEEE) received the B.Eng. and Ph.D. degrees from Xidian University, Xi'an, China, in 2003 and 2009, respectively.

Since 2006, he has been a Teacher with Xidian University, where he was promoted to an Associate Professor and a Full Professor in 2008 and 2010, respectively, both with exceptive admission. He has authored over 100 papers in journals and conferences. He holds over 20 granted patents as a first inventor. He is leading or has completed over 20 projects as the Principle Investigator, funded by the National Natural Science Foundation of China, the National Key Research and Development Program of China, and others. His research interests include computational intelligence, with applications to optimization, learning, data mining, and image understanding.

Dr. Gong is the Executive Committee Member of the Chinese Association for Artificial Intelligence, a Senior Member of the Chinese Computer Federation, and an Associate Editor or an Editorial Board Member of over five journals, including the IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION and the IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS. He was a recipient of the prestigious National Program for Support of the Leading Innovative Talents from the Central Organization Department of China, the Leading Innovative Talent in the Science and Technology from the Ministry of Science and Technology of China, the Excellent Young Scientist Foundation from the National Natural Science Foundation of China, the New Century Excellent Talent from the Ministry of Education of China, and the National Natural Science Award of China.



Zhuping Hu is currently pursuing the Ph.D. degree with Xidian University, Xi'an, China.

Her research interests include deep learning, multiobjective evolution, and network structure optimization.



Wei Zhao (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees from Xidian University, Xi'an, China, in 2002, 2005, and 2015, respectively.

He is currently an Associate Professor with the School of Computer Science and Technology, Xidian University. His research interests include pattern recognition and intelligent systems, with specific interests in attributed graph mining and search, machine learning, signal processing, and precision guiding technology.



Ziyu Guan (Senior Member, IEEE) received the B.S. and Ph.D. degrees in computer science from Zhejiang University, Hangzhou China, in 2004 and 2010, respectively.

He was a Research Scientist with the University of California at Santa Barbara, Santa Barbara, CA, USA, from 2010 to 2012, and a Professor with the School of Information and Technology, Northwest University, Xi'an, China, from 2012 to 2018. He is currently a Professor with the School of Computer Science and Technology, Xidian University, Xi'an. His research interests include attributed graph mining and search, machine learning, expertise modeling and retrieval, and recommender systems.



Mingyang Zhang (Member, IEEE) received the B.Eng. degree in automation and the Ph.D. degree in pattern recognition and intelligent system from Xidian University, Xi'an, China, in 2012 and 2018, respectively.

Since 2018, he has been a Lecturer with the School of Electronic Engineering, Xidian University. His research interests include computational intelligence and remote sensing image understanding.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:41:25 UTC from IEEE Xplore. Restrictions apply.